

```

#!/usr/bin/env python3
"""
КосыТату — Бот инвентаризации студии по плетению косичек
@KosaDH_bot
"""

import os
import sqlite3
import logging
from datetime import datetime
from io import BytesIO

import openpyxl
from openpyxl.styles import Font, PatternFill, Alignment
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import (
    Application, CommandHandler, CallbackQueryHandler,
    MessageHandler, ConversationHandler, filters, ContextTypes
)
from telegram.constants import ParseMode

# — Логирование —————
logging.basicConfig(
    format="%(asctime)s | %(levelname)s | %(message)s",
    level=logging.INFO
)
logger = logging.getLogger(__name__)

# — Настройки —————
TOKEN = os.environ.get("BOT_TOKEN", "REPLACE_WITH_YOUR_TOKEN")
DB_PATH = os.environ.get("DB_PATH", "inventory.db")

CATEGORIES = [
    "Зизи", "Гофре", "Изи Брейд", "Канекалон",
    "Комплект", "Украшения", "Заколки и ободки", "Татуировки",
]

UNIT_MAP = {
    "Зизи": "нитей",
    "Гофре": "нитей",
    "Изи Брейд": "локонов",
    "Канекалон": "пачек",
    "Комплект": "штук",
    "Украшения": "штук",

```

```

        "Заколки и ободки": "штук",
        "Татуировки":      "штук",
    }

```

```

# — Состояния диалога —————

```

```

(
    ST_MAIN,
    ST_INV_CAT, ST_INV_ITEM, ST_INV_QTY,
    ST_ADD_CAT, ST_ADD_NAME, ST_ADD_PHOTO,
    ST_MANAGE,
    ST_REPORT,
) = range(9)

```

```

# =====
#  БАЗА ДАННЫХ
# =====

```

```

def _conn():
    c = sqlite3.connect(DB_PATH)
    c.row_factory = sqlite3.Row
    return c

```

```

def init_db():
    with _conn() as db:
        db.executescript("""
            CREATE TABLE IF NOT EXISTS users (
                tg_id    INTEGER PRIMARY KEY,
                username TEXT,
                name     TEXT,
                role     TEXT NOT NULL DEFAULT 'pending',
                joined   TEXT NOT NULL
            );

            CREATE TABLE IF NOT EXISTS items (
                id        INTEGER PRIMARY KEY AUTOINCREMENT,
                category  TEXT NOT NULL,
                name      TEXT NOT NULL,
                photo_id  TEXT,
                unit      TEXT NOT NULL,
                added_at  TEXT NOT NULL
            );

            CREATE TABLE IF NOT EXISTS sessions (
                id        INTEGER PRIMARY KEY AUTOINCREMENT,
                user_id   INTEGER NOT NULL,
                started_at TEXT NOT NULL,
                finished_at TEXT,

```

```

        FOREIGN KEY(user_id) REFERENCES users(tg_id)
    );

    CREATE TABLE IF NOT EXISTS entries (
        id            INTEGER PRIMARY KEY AUTOINCREMENT,
        session_id    INTEGER NOT NULL,
        item_id       INTEGER NOT NULL,
        quantity      REAL     NOT NULL,
        noted_at      TEXT     NOT NULL,
        FOREIGN KEY(session_id) REFERENCES sessions(id),
        FOREIGN KEY(item_id)   REFERENCES items(id)
    );
    """

# — Пользователи —————
def db_user(tg_id):
    with _conn() as db:
        return db.execute("SELECT * FROM users WHERE tg_id=?", (tg_id,)).fetchone()

def db_count_admins():
    with _conn() as db:
        return db.execute("SELECT COUNT(*) FROM users WHERE role='admin']").fetchone()

def db_upsert_user(tg_id, username, name, role="pending"):
    with _conn() as db:
        db.execute("""
            INSERT INTO users(tg_id,username,name,role,joined)
            VALUES(?,?,?,?,:)
            ON CONFLICT(tg_id) DO UPDATE
            SET username=excluded.username, name=excluded.name
            """, (tg_id, username or "", name, role, now()))

def db_set_role(tg_id, role):
    with _conn() as db:
        db.execute("UPDATE users SET role=? WHERE tg_id=?", (role, tg_id))

def db_admins():
    with _conn() as db:
        return db.execute(
            "SELECT * FROM users WHERE role IN ('admin','director')"
        ).fetchall()

def db_pending():
    with _conn() as db:
        return db.execute("SELECT * FROM users WHERE role='pending']").fetchall()

# — Товары —————

```

```

def db_items(category):
    with _conn() as db:
        return db.execute(
            "SELECT * FROM items WHERE category=? ORDER BY name", (category,)
        ).fetchall()

def db_item(item_id):
    with _conn() as db:
        return db.execute("SELECT * FROM items WHERE id=?", (item_id,)).fetchone()

def db_add_item(category, name, photo_id, unit):
    with _conn() as db:
        db.execute(
            "INSERT INTO items(category,name,photo_id,unit,added_at) VALUES(?,?,?,?
            (category, name, photo_id, unit, now())
        )

def db_delete_item(item_id):
    with _conn() as db:
        db.execute("DELETE FROM items WHERE id=?", (item_id,))

# — Инвентаризации —————
def db_new_session(user_id):
    with _conn() as db:
        c = db.execute(
            "INSERT INTO sessions(user_id,started_at) VALUES(?,?)", (user_id, now
        )
        return c.lastrowid

def db_close_session(sid):
    with _conn() as db:
        db.execute("UPDATE sessions SET finished_at=? WHERE id=?", (now(), sid))

def db_add_entry(sid, item_id, qty):
    with _conn() as db:
        db.execute(
            "INSERT INTO entries(session_id,item_id,quantity,noted_at) VALUES(?,?,?
            (sid, item_id, qty, now())
        )

def db_session_entries(sid):
    with _conn() as db:
        return db.execute("""
            SELECT e.quantity, i.name, i.category, i.unit
            FROM entries e JOIN items i ON i.id=e.item_id
            WHERE e.session_id=?
            """, (sid,)).fetchall()

```

```

def db_latest_per_item():
    """Последний остаток по каждой позиции."""
    with _conn() as db:
        return db.execute("""
            SELECT i.category, i.name, i.unit, e.quantity,
                   s.finished_at, u.name AS user_name
            FROM entries e
            JOIN items i    ON i.id=e.item_id
            JOIN sessions s ON s.id=e.session_id
            JOIN users u    ON u.tg_id=s.user_id
            WHERE s.finished_at IS NOT NULL
            AND e.id = (
                SELECT MAX(e2.id) FROM entries e2
                JOIN sessions s2 ON s2.id=e2.session_id
                WHERE e2.item_id=e.item_id AND s2.finished_at IS NOT NULL
            )
            ORDER BY i.category, i.name
        """).fetchall()

```

```

def db_last_two_sessions():
    with _conn() as db:
        return db.execute("""
            SELECT s.id, s.finished_at, u.name AS user_name
            FROM sessions s JOIN users u ON u.tg_id=s.user_id
            WHERE s.finished_at IS NOT NULL
            ORDER BY s.finished_at DESC LIMIT 2
        """).fetchall()

```

```

def db_sessions_for_excel():
    with _conn() as db:
        return db.execute("""
            SELECT s.id, s.finished_at, u.name AS user_name
            FROM sessions s JOIN users u ON u.tg_id=s.user_id
            WHERE s.finished_at IS NOT NULL
            ORDER BY s.finished_at DESC LIMIT 30
        """).fetchall()

```

```

# =====
#  ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ
#  =====

```

```

def now():    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")
def today():  return datetime.now().strftime("%d.%m.%Y %H:%M")
def qstr(q):  return str(int(q)) if q == int(q) else str(q)

```

```

def is_staff(u): return u and u["role"] in ("admin", "director", "employee")

```

```
def is_mgmt(u): return u and u["role"] in ("admin", "director")
```

```
ROLE_LABEL = {
    "admin": "👑 Администратор",
    "director": "🏢 Директор",
    "employee": "👤 Сотрудник",
    "pending": "⌚ Ожидает одобрения",
}
```

```
def main_kb(role):
    btns = [[InlineKeyboardButton("📦 Начать инвентаризацию", callback_data="inv_
if role in ("admin", "director"):
    btns += [
        [InlineKeyboardButton("+ Добавить товар", callback_data="add_i
        [InlineKeyboardButton("👥 Управление доступом", callback_data="manag
    ]
    btns.append([InlineKeyboardButton("📊 Отчёты", callback_data="report")])
    return InlineKeyboardMarkup(btns)
```

```
def cat_kb(prefix):
    rows = []
    for i in range(0, len(CATEGORIES), 2):
        row = [InlineKeyboardButton(CATEGORIES[i], callback_data=f"{prefix}:{CATE
        if i + 1 < len(CATEGORIES):
            row.append(InlineKeyboardButton(CATEGORIES[i+1], callback_data=f"{pre
        rows.append(row)
    rows.append([InlineKeyboardButton("⬅️ Главное меню", callback_data="back_main
    return InlineKeyboardMarkup(rows)
```

```
def items_kb(category, entries: dict):
    items = db_items(category)
    btns = []
    for it in items:
        label = it["name"]
        if it["id"] in entries:
            label += f" ✅ {entries[it['id']]}"
        btns.append([InlineKeyboardButton(label, callback_data=f"inv_item:{it['id
    btns.append([InlineKeyboardButton("⬅️ К категориям", callback_data="
    btns.append([InlineKeyboardButton("✅ Завершить инвентаризацию", callback_dat
    return InlineKeyboardMarkup(btns)
```

```
def report_kb():
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("📋 Текущие остатки", callback_data="rep_curr
        [InlineKeyboardButton("📉 Разница (2 последних)", callback_data="rep_diff
        [InlineKeyboardButton("📄 Скачать Excel", callback_data="rep_exce
        [InlineKeyboardButton("⬅️ Главное меню", callback_data="back_mai
```

```

    })

def back_report_kb():
    return InlineKeyboardMarkup([[InlineKeyboardButton("🔍 К отчётам", callback_data="back_report_kb")]])

# =====
# ХЭНДЛЕРЫ
# =====

# — /start —————
async def cmd_start(upd: Update, ctx: ContextTypes.DEFAULT_TYPE):
    u = upd.effective_user
    row = db_user(u.id)

    if not row:
        # Первый пользователь автоматически становится администратором
        role = "admin" if db_count_admins() == 0 else "pending"
        db_upsert_user(u.id, u.username, u.full_name, role)
        row = db_user(u.id)

    if role == "admin":
        await upd.message.reply_text(
            f"👑 Добро пожаловать, Администратор!\n\n"
            "Вы первый пользователь и получили роль Администратора автоматически\n\n"
            "Теперь вы можете добавлять товары, управлять доступом и проводить аудит\n\n"
            "Ваш ID: " + str(u.id)
        )
        return ST_MAIN

    # Уведомляем администраторов
    for admin in db_admins():
        try:
            await ctx.bot.send_message(
                admin["tg_id"],
                f"🆕 *Запрос доступа*\n\n"
                f"👤 {u.full_name}\n\n"
                f"📱 @{u.username or '-'}\n\n"
                f"🆔 `{u.id}`",
                parse_mode=ParseMode.MARKDOWN,
                reply_markup=InlineKeyboardMarkup([
                    [
                        InlineKeyboardButton("✅ Сотрудник", callback_data=f"approve_{u.id}_employee"),
                        InlineKeyboardButton("🏢 Директор", callback_data=f"approve_{u.id}_director"),
                    ],
                    [
                        InlineKeyboardButton("❌ Отклонить", callback_data=f"approve_{u.id}_deny"),
                    ]
                ])
            )
        except:
            pass

```

```

except Exception as e:
    logger.warning(f"Не удалось уведомить {admin['tg_id']}: {e}")

await upd.message.reply_text(
    "👋 Добро пожаловать в бот инвентаризации *КосыТату*!\n\n"
    "⌚ Ваш запрос отправлен администратору. Ожидайте одобрения — "
    "вам придёт уведомление.",
    parse_mode=ParseMode.MARKDOWN
)
return ConversationHandler.END

role = row["role"]
if role == "pending":
    await upd.message.reply_text("⌚ Ваш запрос ещё не одобрен. Пожалуйста, п
    return ConversationHandler.END
if role == "rejected":
    await upd.message.reply_text("❌ В доступе отказано. Обратитесь к админис
    return ConversationHandler.END

await upd.message.reply_text(
    f"Привет, {u.first_name}! {ROLE_LABEL.get(role, role)}\n\nЧто будем делат
    reply_markup=main_kb(role)
)
return ST_MAIN

# — Одобрение пользователей (работает вне диалога тоже) —————
async def cb_approve(upd: Update, ctx: ContextTypes.DEFAULT_TYPE):
    q = upd.callback_query
    approver = db_user(q.from_user.id)
    if not is_mgmt(approver):
        await q.answer("❌ Нет прав", show_alert=True)
        return

    parts = q.data.split(":")
    new_role, tid = parts[1], int(parts[2])
    db_set_role(tid, new_role)
    await q.answer()

    labels = {"employee": "Сотрудник ✅", "director": "Директор 🏢", "rejected":
    await q.edit_message_text(f"Готово! Роль назначена: {labels.get(new_role, new

msg = ("❌ В доступе отказано. Обратитесь к администратору." if new_role == "
    else f"✅ Ваш доступ одобрен!\nРоль: {labels[new_role]}\n\nНапишите /s
try:
    await ctx.bot.send_message(tid, msg)
except Exception:

```


pass

```
# — Главное меню —————
async def cb_main(upd: Update, ctx: ContextTypes.DEFAULT_TYPE):
    q = upd.callback_query
    u = db_user(q.from_user.id)
    await q.answer()

    if q.data == "back_main":
        if not is_staff(u):
            return ConversationHandler.END
        await q.edit_message_text("Главное меню:", reply_markup=main_kb(u["role"])
        return ST_MAIN

    if q.data == "inv_start":
        if not is_staff(u):
            return ST_MAIN
        sid = db_new_session(q.from_user.id)
        ctx.user_data.update({"sid": sid, "inv": {}, "cat": None})
        await q.edit_message_text("📦 Выберите категорию:", reply_markup=cat_kb("
        return ST_INV_CAT

    if q.data == "add_item":
        if not is_mgmt(u):
            return ST_MAIN
        await q.edit_message_text("✚ Добавить товар\n\nВыберите категорию:", rep
        return ST_ADD_CAT

    if q.data == "manage_users":
        if not is_mgmt(u):
            return ST_MAIN
        return await show_manage(q, ctx)

    if q.data == "report":
        await q.edit_message_text("📊 Выберите отчёт:", reply_markup=report_kb())
        return ST_REPORT

    return ST_MAIN

# =====
# ИНВЕНТАРИЗАЦИЯ
# =====

async def cb_inv_cat(upd: Update, ctx: ContextTypes.DEFAULT_TYPE):
    q = upd.callback_query; await q.answer()
```

```

if q.data == "back_main":
    u = db_user(q.from_user.id)
    await q.edit_message_text("Главное меню:", reply_markup=main_kb(u["role"])
    return ST_MAIN

if q.data == "back_cats":
    await q.edit_message_text("📦 Выберите категорию:", reply_markup=cat_kb("
    return ST_INV_CAT

if q.data == "inv_finish":
    return await do_finish(q, ctx)

cat = q.data.split(":", 1)[1]
ctx.user_data["cat"] = cat
items = db_items(cat)

if not items:
    await q.edit_message_text(
        f"📁 *{cat}*\n\n⚠ В этой категории пока нет товаров.\n"
        f"Администратор может добавить их через «➕ Добавить товар».",
        parse_mode=ParseMode.MARKDOWN,
        reply_markup=InlineKeyboardMarkup([
            [InlineKeyboardButton("⬅ К категориям", callback_data="
            [InlineKeyboardButton("✅ Завершить инвентаризацию", callback_dat
        ])
    )
    return ST_INV_CAT

await q.edit_message_text(
    f"📁 *{cat}*\n\nВыберите позицию:\n_✅ - уже заполнено_",
    parse_mode=ParseMode.MARKDOWN,
    reply_markup=items_kb(cat, ctx.user_data.get("inv", {}))
)
return ST_INV_ITEM

async def cb_inv_item(upd: Update, ctx: ContextTypes.DEFAULT_TYPE):
    q = upd.callback_query; await q.answer()

    if q.data == "back_cats":
        await q.edit_message_text("📦 Выберите категорию:", reply_markup=cat_kb("
        return ST_INV_CAT

    if q.data == "inv_finish":
        return await do_finish(q, ctx)

```

```

item_id = int(q.data.split(":")[1])
item = db_item(item_id)
ctx.user_data["item_id"] = item_id

caption = (
    f"📌 *{item['name']}*\n"
    f"📁 {item['category']}\n"
    f"📏 Единица учёта: *{item['unit']}*\n\n"
    f"Введите количество:"
)

back_btn = InlineKeyboardMarkup([[InlineKeyboardButton("⬅️ Назад", callback_c

if item["photo_id"]:
    try:
        await q.message.delete()
    except Exception:
        pass
    sent = await ctx.bot.send_photo(
        q.from_user.id, item["photo_id"],
        caption=caption, parse_mode=ParseMode.MARKDOWN, reply_markup=back_btn
    )
    ctx.user_data["photo_msg_id"] = sent.message_id
else:
    await q.edit_message_text(caption, parse_mode=ParseMode.MARKDOWN, reply_m

return ST_INV_QTY

async def cb_back_to_cat(upd: Update, ctx: ContextTypes.DEFAULT_TYPE):
    """Возврат из ввода количества к списку позиций."""
    q = upd.callback_query; await q.answer()
    cat = ctx.user_data.get("cat", "")

    # Удаляем сообщение с фото если было
    photo_mid = ctx.user_data.pop("photo_msg_id", None)
    if photo_mid:
        try:
            await ctx.bot.delete_message(q.from_user.id, photo_mid)
        except Exception:
            pass

    await ctx.bot.send_message(
        q.from_user.id,
        f"📁 *{cat}*\n\nВыберите позицию:",
        parse_mode=ParseMode.MARKDOWN,
        reply_markup=items_kb(cat, ctx.user_data.get("inv", {}))
    )

```

```

return ST_INV_ITEM

async def msg_qty(upd: Update, ctx: ContextTypes.DEFAULT_TYPE):
    text = upd.message.text.replace(",", ".").strip()
    try:
        qty = float(text)
        if qty < 0:
            raise ValueError
    except ValueError:
        await upd.message.reply_text(
            "⚠ Введите число.\n"
            "Примеры: *42* или *0.5* (для половины пачки)",
            parse_mode=ParseMode.MARKDOWN
        )
    return ST_INV_QTY

item_id = ctx.user_data["item_id"]
item     = db_item(item_id)
sid      = ctx.user_data["sid"]

db_add_entry(sid, item_id, qty)
ctx.user_data["inv"][item_id] = f"{qstr(qty)} {item['unit']}"

# Удаляем фото-сообщение если было
photo_mid = ctx.user_data.pop("photo_msg_id", None)
if photo_mid:
    try:
        await ctx.bot.delete_message(upd.effective_user.id, photo_mid)
    except Exception:
        pass

cat = ctx.user_data["cat"]
await upd.message.reply_text(f"✅ Записано: *{item['name']}* — {qstr(qty)} {i
                                parse_mode=ParseMode.MARKDOWN)
await upd.message.reply_text(
    f"📁 *{cat}*\n\nПродолжайте или завершайте:",
    parse_mode=ParseMode.MARKDOWN,
    reply_markup=items_kb(cat, ctx.user_data["inv"])
)
return ST_INV_ITEM

async def do_finish(q, ctx: ContextTypes.DEFAULT_TYPE):
    sid = ctx.user_data.get("sid")
    if sid:
        db_close_session(sid)

```

```

inv = ctx.user_data.get("inv", {})
u = db_user(q.from_user.id)
name = u["name"] if u else "Неизвестно"

# Собираем итог по категориям
cat_data: dict[str, list] = {}
with _conn() as db:
    for item_id, qty_label in inv.items():
        it = db.execute("SELECT * FROM items WHERE id=?", (item_id,)).fetchone()
        if it:
            cat_data.setdefault(it["category"], []).append(
                f" • {it['name']}: {qty_label}"
            )

lines = [f"📦 *Инвентаризация завершена*", f"👤 {name}", f"🕒 {today()}\n"]
if cat_data:
    for cat, rows in cat_data.items():
        lines.append(f"*{cat}:*")
        lines.extend(rows)
        lines.append("")
else:
    lines.append("_ (позиции не были заполнены) _")

summary = "\n".join(lines)
await q.message.reply_text(summary, parse_mode=ParseMode.MARKDOWN)

# Уведомляем других администраторов и директоров
for admin in db_admins():
    if admin["tg_id"] != q.from_user.id:
        try:
            await ctx.bot.send_message(
                admin["tg_id"],
                f"🔔 *Новая инвентаризация*\n\n{summary}",
                parse_mode=ParseMode.MARKDOWN
            )
        except Exception:
            pass

db_u = db_user(q.from_user.id)
await q.message.reply_text("Главное меню:", reply_markup=main_kb(db_u["role"])
ctx.user_data.clear()
return ST_MAIN

```

#

ДОБАВЛЕНИЕ ТОВАРА

#

```
async def cb_add_cat(upd: Update, ctx: ContextTypes.DEFAULT_TYPE):
    q = upd.callback_query; await q.answer()
    if q.data == "back_main":
        u = db_user(q.from_user.id)
        await q.edit_message_text("Главное меню:", reply_markup=main_kb(u["role"])
        return ST_MAIN
    cat = q.data.split(":", 1)[1]
    ctx.user_data["new_cat"] = cat
    ctx.user_data["new_unit"] = UNIT_MAP[cat]
    await q.edit_message_text(
        f"📁 *{cat}*\n\n"
        f"Введите название (цвет) нового товара.\n\n"
        f"_Примеры:_\n"
        f"• Однотонный розовый\n"
        f"• Двухцветный чёрный/розовый\n"
        f"• Трёхцветный синий/белый/фиолетовый",
        parse_mode=ParseMode.MARKDOWN
    )
    return ST_ADD_NAME
```

```
async def msg_add_name(upd: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data["new_name"] = upd.message.text.strip()
    await upd.message.reply_text(
        f"📷 Отправьте фото для *{ctx.user_data['new_name']}*\n\n"
        f"Или нажмите кнопку чтобы пропустить:",
        parse_mode=ParseMode.MARKDOWN,
        reply_markup=InlineKeyboardMarkup([
            [InlineKeyboardButton("➡️ Пропустить (без фото)", callback_data="skip
        ]
    )
    return ST_ADD_PHOTO
```

```
async def msg_add_photo(upd: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data["new_photo"] = upd.message.photo[-1].file_id
    return await _save_item(upd, ctx)
```

```
async def cb_skip_photo(upd: Update, ctx: ContextTypes.DEFAULT_TYPE):
    q = upd.callback_query; await q.answer()
    ctx.user_data["new_photo"] = None
    return await _save_item(upd, ctx)
```



```

        ),
    ],
]

else:
    text = "👥 Нет заявок на рассмотрении."
    btns = []
    btns.append([InlineKeyboardButton("🏠 Главное меню", callback_data="back_main")])
    await q.edit_message_text(text, parse_mode=ParseMode.MARKDOWN,
                             reply_markup=InlineKeyboardMarkup(btns))
    return ST_MANAGE

async def cb_manage(upd: Update, ctx: ContextTypes.DEFAULT_TYPE):
    q = upd.callback_query; await q.answer()
    if q.data == "back_main":
        u = db_user(q.from_user.id)
        await q.edit_message_text("Главное меню:", reply_markup=main_kb(u["role"]))
        return ST_MAIN
    if q.data.startswith("apr:"):
        await cb_approve(upd, ctx)
        # Обновляем список
        pending = db_pending()
        if pending:
            return await show_manage(q, ctx)
        u = db_user(q.from_user.id)
        await ctx.bot.send_message(
            q.from_user.id, "👥 Больше нет заявок.", reply_markup=main_kb(u["role"])
        )
        return ST_MAIN
    return ST_MANAGE

# =====
# ОТЧЁТЫ
# =====

async def cb_report(upd: Update, ctx: ContextTypes.DEFAULT_TYPE):
    q = upd.callback_query; await q.answer()
    data = q.data

    if data == "back_main":
        u = db_user(q.from_user.id)
        await q.edit_message_text("Главное меню:", reply_markup=main_kb(u["role"]))
        return ST_MAIN

    if data == "report":
        await q.edit_message_text("📊 Выберите отчёт:", reply_markup=report_kb())

```



```

    return ST_REPORT

# — Текущие остатки —————
if data == "rep_current":
    rows = db_latest_per_item()
    if not rows:
        await q.edit_message_text(
            "📦 Данных пока нет.\nПроведите первую инвентаризацию!",
            reply_markup=back_report_kb()
        )
        return ST_REPORT

    lines = ["📦 *Текущие остатки*\n"]
    cur_cat = None
    for r in rows:
        if r["category"] != cur_cat:
            cur_cat = r["category"]
            lines.append(f"\n*{cur_cat}:*")
        lines.append(f"    • {r['name']}: {qstr(r['quantity'])} {r['unit']}")

    await q.edit_message_text(
        "\n".join(lines), parse_mode=ParseMode.MARKDOWN, reply_markup=back_re
    )
    return ST_REPORT

# — Разница —————
if data == "rep_diff":
    sessions = db_last_two_sessions()
    if len(sessions) < 2:
        await q.edit_message_text(
            "⚠️ Для сравнения нужно минимум *2* завершённые инвентаризации.",
            parse_mode=ParseMode.MARKDOWN, reply_markup=back_report_kb()
        )
        return ST_REPORT

    newer, older = sessions[0], sessions[1]
    ne = {e["name"]: e for e in db_session_entries(newer["id"])}
    oe = {e["name"]: e for e in db_session_entries(older["id"])}
    all_names = sorted(set(ne) | set(oe))

    lines = [
        "📊 *Разница по остаткам*\n",
        f"Старая: {older['user_name']} ({older['finished_at'][:10]})",
        f"Новая: {newer['user_name']} ({newer['finished_at'][:10]})\n",
    ]

    changed = False
    for name in all_names:

```

```

        nq = ne[name]["quantity"] if name in ne else 0.0
        oq = oe[name]["quantity"] if name in oe else 0.0
        diff = nq - oq
        if diff == 0:
            continue
        changed = True
        unit = (ne.get(name) or oe.get(name))["unit"]
        sign = "+" if diff > 0 else ""
        emoji = "▲" if diff > 0 else "▼"
        lines.append(f"{emoji} {name}: {sign}{qstr(diff)} {unit}")

    if not changed:
        lines.append("✅ Изменений нет — остатки совпадают")

    await q.edit_message_text(
        "\n".join(lines), parse_mode=ParseMode.MARKDOWN, reply_markup=back_re
    )
    return ST_REPORT

# — Excel —————
if data == "rep_excel":
    sessions = db_sessions_for_excel()
    wb = openpyxl.Workbook()
    ws = wb.active
    ws.title = "Инвентаризация"

    hfont = Font(bold=True, color="FFFFFF")
    hfill = PatternFill("solid", fgColor="1B4F72")
    halign = Alignment(horizontal="center", vertical="center")

    headers = ["Дата", "Сотрудник", "Категория", "Наименование", "Количество"]
    for col, h in enumerate(headers, 1):
        cell = ws.cell(row=1, column=col, value=h)
        cell.font = hfont; cell.fill = hfill; cell.alignment = halign

    row_n = 2
    for s in sessions:
        for e in db_session_entries(s["id"]):
            ws.cell(row=row_n, column=1, value=s["finished_at"][:16])
            ws.cell(row=row_n, column=2, value=s["user_name"])
            ws.cell(row=row_n, column=3, value=e["category"])
            ws.cell(row=row_n, column=4, value=e["name"])
            ws.cell(row=row_n, column=5, value=e["quantity"])
            ws.cell(row=row_n, column=6, value=e["unit"])
            row_n += 1

    for col in ws.columns:

```

```

        width = max(len(str(cell.value or "")) for cell in col)
        ws.column_dimensions[col[0].column_letter].width = min(width + 4, 45)

    buf = BytesIO(); wb.save(buf); buf.seek(0)
    fname = f"inventory_{datetime.now().strftime('%Y%m%d_%H%M')}.xlsx"
    await ctx.bot.send_document(
        q.from_user.id, document=buf, filename=fname,
        caption=f" Отчёт по инвентаризации\n{today()}"
    )
    await q.edit_message_text("✅ Excel-файл отправлен выше ", reply_markup=
    return ST_REPORT

return ST_REPORT

# =====
# ЗАПУСК
# =====

def main():
    init_db()
    logger.info("База данных инициализирована")

    app = Application.builder().token(TOKEN).build()

    # Глобальный обработчик одобрений (работает в любом состоянии диалога)
    app.add_handler(CallbackQueryHandler(cb_approve, pattern=r"^apr:"))

    conv = ConversationHandler(
        entry_points=[CommandHandler("start", cmd_start)],
        states={
            ST_MAIN: [
                CallbackQueryHandler(cb_main),
            ],
            ST_INV_CAT: [
                CallbackQueryHandler(
                    cb_inv_cat,
                    pattern=r"^(inv_cat:|back_cats$|back_main$|inv_finish$)"
                ),
            ],
            ST_INV_ITEM: [
                CallbackQueryHandler(
                    cb_inv_item,
                    pattern=r"^(inv_item:|back_cats$|inv_finish$)"
                ),
            ],
            ST_INV_QTY: [

```

```

        MessageHandler(filters.TEXT & ~filters.COMMAND, msg_qty),
        CallbackQueryHandler(cb_back_to_cat, pattern=r"^back_to_cat$"),
    ],
    ST_ADD_CAT: [
        CallbackQueryHandler(cb_add_cat, pattern=r"^(add_cat:|back_main$)
    ],
    ST_ADD_NAME: [
        MessageHandler(filters.TEXT & ~filters.COMMAND, msg_add_name),
    ],
    ST_ADD_PHOTO: [
        MessageHandler(filters.PHOTO, msg_add_photo),
        CallbackQueryHandler(cb_skip_photo, pattern=r"^skip_photo$"),
    ],
    ST_MANAGE: [
        CallbackQueryHandler(cb_manage),
    ],
    ST_REPORT: [
        CallbackQueryHandler(cb_report),
    ],
    },
    fallbacks=[CommandHandler("start", cmd_start)],
    per_user=True,
    per_chat=True,
    allow_reentry=True,
)

app.add_handler(conv)

logger.info("Бот запущен!")
app.run_polling(allowed_updates=Update.ALL_TYPES)

if __name__ == "__main__":
    main()

```